

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»

Інститут комп'ютерних систем
Кафедра інформаційних систем

Костянтин ШИДЛЮХ
(група AI-232)

ЛАБОРАТОРНА РОБОТА №4

З дисципліни «Програмування мобільних пристроїв на базі ОС Android»

Тема: «Локальна база даних Room та Repository в Android-застосунку»

Спеціальність:

122 Комп'ютерні науки

Освітньо-професійна програма:

Комп'ютерні науки

Завдання (варіант №4): треба зробити додаток «Мої завдання».

ХІД РОБОТИ

Крок 1: Налаштування проєкту та залежностей

1. Підключення бібліотек: У файл `build.gradle.kts` модуля `app` додано залежності для роботи з бібліотекою Room (`room-runtime`, `room-ktx`, `room-compiler`).
2. Налаштування компілятора: Підключено сучасний плагін обробки анотацій Kotlin Symbol Processing (KSP) через систему каталогів `libs.versions.toml`.
3. Синхронізація версій: Забезпечено строгу відповідність версії плагіна KSP до версії компілятора Kotlin для уникнення помилок генерації класів під час збірки проєкту (вирішено проблему `IncompatibleClassChangeError`).

Крок 2: Проєктування шару локальних даних (Data Layer)

1. Створення Entity-класу: Створено клас `TaskEntity`, який репрезентує таблицю `tasks` у базі даних. Поле `id` налаштовано як первинний ключ із функцією автогенерації (`@PrimaryKey(autoGenerate = true)`).
2. Конвертація даних (Mapper): Оскільки застосунок використовує складні типи даних для UI (наприклад, `Enum` для пріоритетів `TaskPriority`), реалізовано функції-мапери (`toDomain()` та `toEntity()`) для зручного перетворення сутностей бази даних у бізнес-моделі і навпаки.
3. Реалізація DAO: Створено інтерфейс `TaskDao` з SQL-запитами для виконання базових CRUD-операцій:
4. Вставлення (`@Insert`) та оновлення (`@Update`) завдань.
5. Видалення (`@Delete`) об'єктів.
6. Отримання списку всіх завдань у вигляді асинхронного потоку `Flow`.

7. Впорядкування: Реалізовано кастомне сортування завдань за пріоритетом (High, Medium, Low) за допомогою SQL-конструкції CASE та додаткове сортування за id для однакових пріоритетів.
8. Ініціалізація бази даних: Створено абстрактний клас AppDatabase, що наслідує RoomDatabase. Для забезпечення єдиного екземпляра бази даних на весь застосунок реалізовано патерн Singleton у об'єкті DatabaseProvider.

Крок 3. Побудова архітектури (Clean Architecture та MVVM)

1. Створення Repository: Реалізовано клас TaskRepository, який виступає єдиним джерелом істини (Single Source of Truth) для застосунку. Репозиторій інкапсулює логіку роботи з DAO і повертає вже готові доменні моделі TaskItem.
2. Оновлення ViewModel: TasksViewModel переписано для роботи з репозиторієм замість локальних колекцій у пам'яті. Використано оператор stateIn для перетворення Flow з репозиторію у стейт-голдер StateFlow, що гарантує збереження даних при зміні конфігурації екрана.
3. Створення ViewModel Factory: Для коректної передачі екземпляра TaskRepository у конструктор ViewModel розроблено фабрику TasksViewModelFactory.

Крок 4. Адаптація користувацького інтерфейсу (UI Layer)

1. Навігація: У кореневому компоненті навігації (TaskNavigation) налаштовано створення екземплярів AppDatabase, TaskRepository та TasksViewModel із подальшою передачею ViewModel до всіх екранів застосунку.
2. Екран списку: UI адаптовано для зчитування даних безпосередньо з бази через collectAsState(). Після перезапуску застосунку раніше додані дані успішно завантажуються з пам'яті пристрою.

3. Екран додавання/редагування:
4. Оновлено логіку збереження об'єктів (Room автоматично генерує id для нових завдань).
5. Реалізовано миттєве заповнення полів форми при переході в режим редагування за допомогою блоку LaunchedEffect, який реагує на асинхронне завантаження даних конкретного завдання з БД.
6. Екран деталей: Змінено логіку видалення та зміни статусу завдання (прапорець isCompleted) – тепер зміни фіксуються через репозиторій з викликом методу update або delete у Room.

ДЕМОНСТРАЦІЯ РОБОТИ

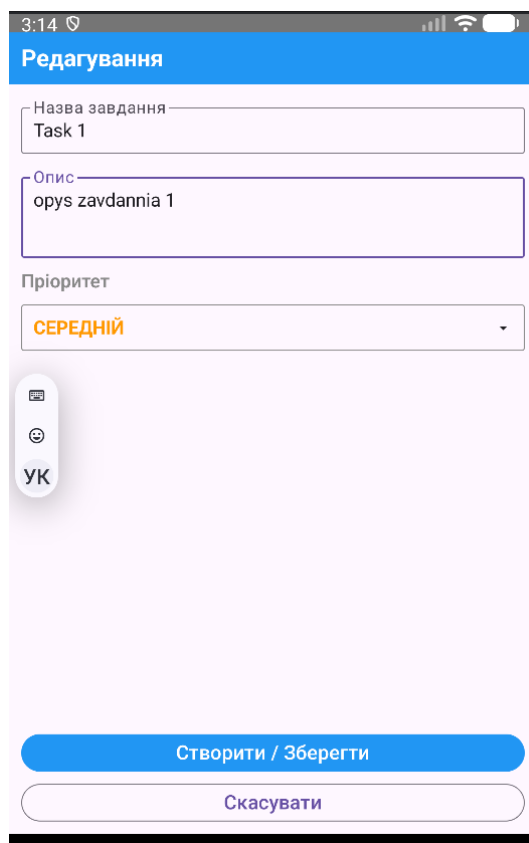


Рисунок 1 – Додаємо перше завдання

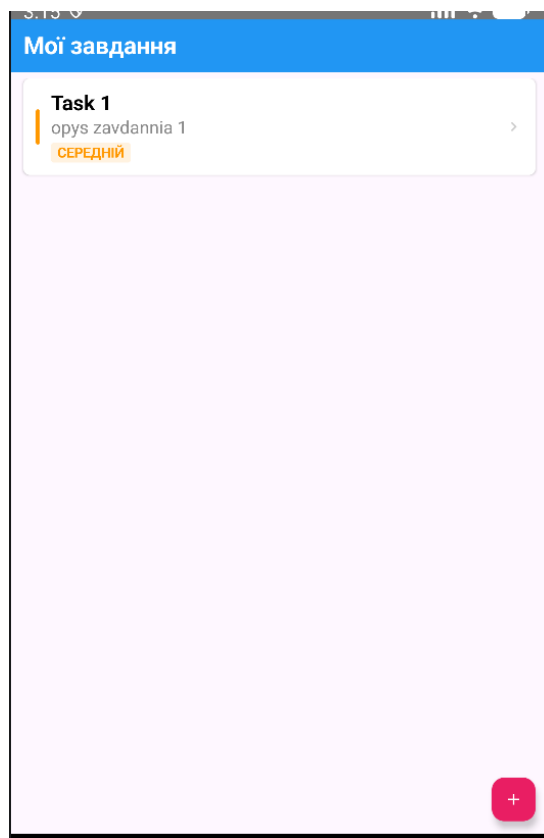


Рисунок 2 – Додано перше завдання (також показ головного меню)

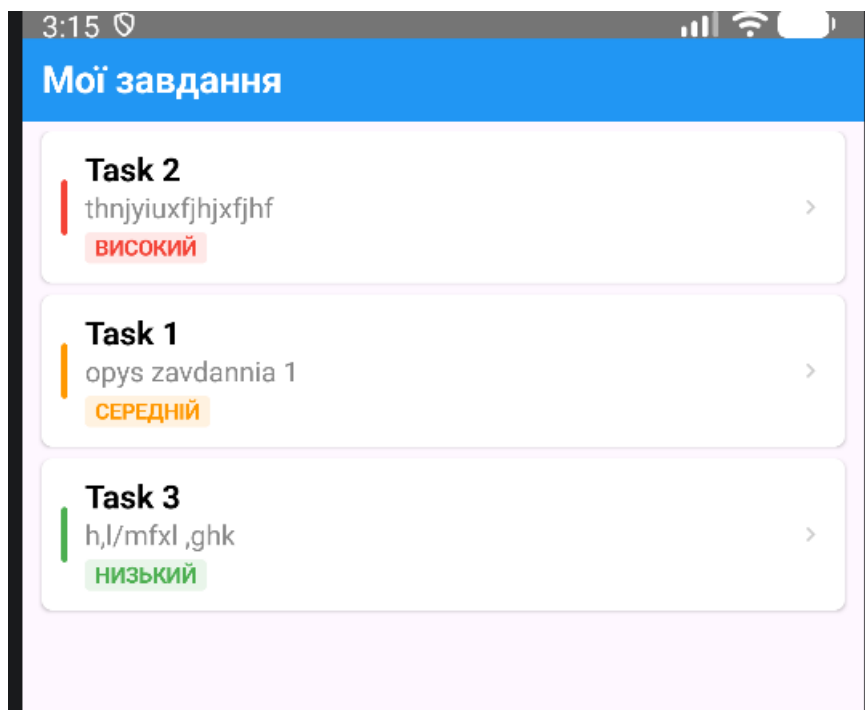


Рисунок 3 – Додано ще кілька завдань (після створення/редагування проводиться впорядкування завдань за їхньою пріорітетністю)

The screenshot shows a mobile application interface for editing a task. At the top, the status bar displays the time 3:16 and signal indicators. The app's header is a blue bar with the title 'Редагування'. Below the header, there are three input fields: 'Назва завдання' (Task Name) containing 'Task 1', 'Опис' (Description) containing 'opys zavdannia 1 (blablablabla)', and 'Пріоритет' (Priority) with a dropdown menu showing 'СЕРЕДНІЙ' (Medium). To the left of the main form is a vertical sidebar with three icons: a calendar, a smiley face, and a button labeled 'УК'. At the bottom of the screen is a blue button with the text 'Створити / Зберегти' (Create / Save).

Рисунок 4 – Показ попереднього завдання (та його редагування)

The screenshot shows the 'Деталі завдання' (Task Details) screen. The status bar at the top shows the time 3:27. The app's header is a blue bar with the title 'Деталі завдання'. The main content area has a light pink background and contains a task card for 'Task 1'. The card displays 'До завтра' (Tomorrow) and a yellow 'СЕРЕДНІЙ' (Medium) priority tag. Below the card is a text field for 'Опис' (Description) with the text 'opys zavdannia 1 (blablablabla)'. Under the description is a checkbox labeled 'Позначити як виконане' (Mark as completed), which is currently checked. At the bottom of the screen are three buttons: a red 'Редагувати' (Edit) button, a white 'Видалити' (Delete) button, and a grey 'Назад' (Back) button.

Рисунок 5 – Редагування завдання

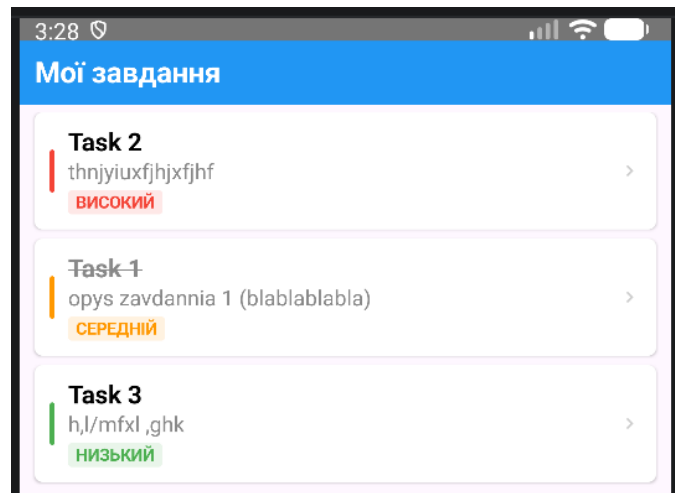


Рисунок 6 – Показ меню з відредагованим завданням (також поставлено мітку «Позначити як виконане»)

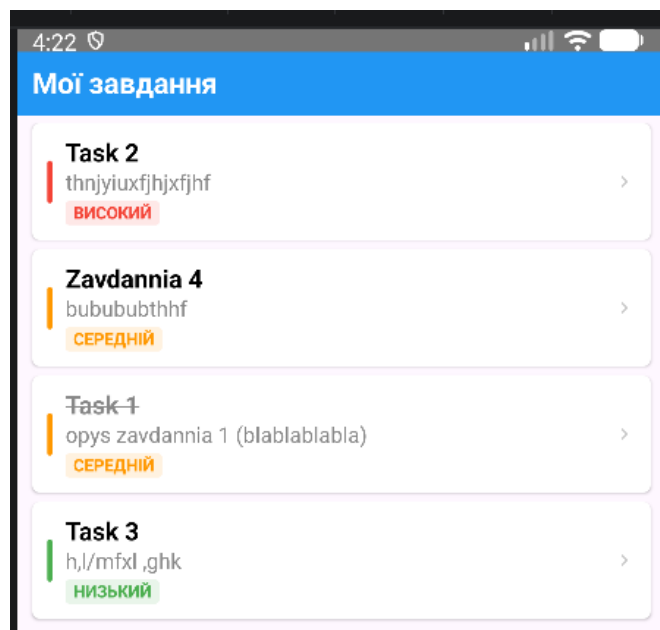
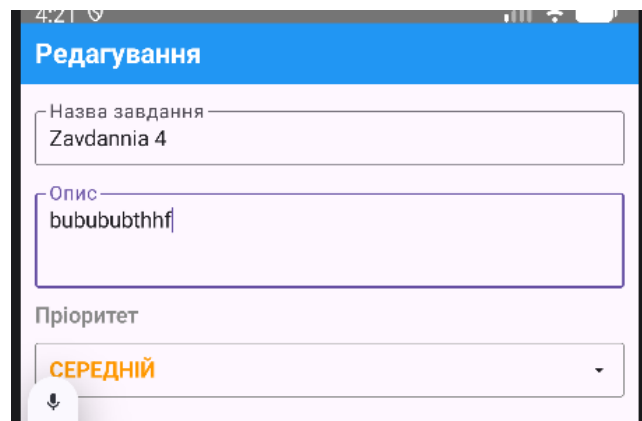


Рисунок 7, 8 – Додавання нового завдання для демонстрації впорядкування за пріоритетністю

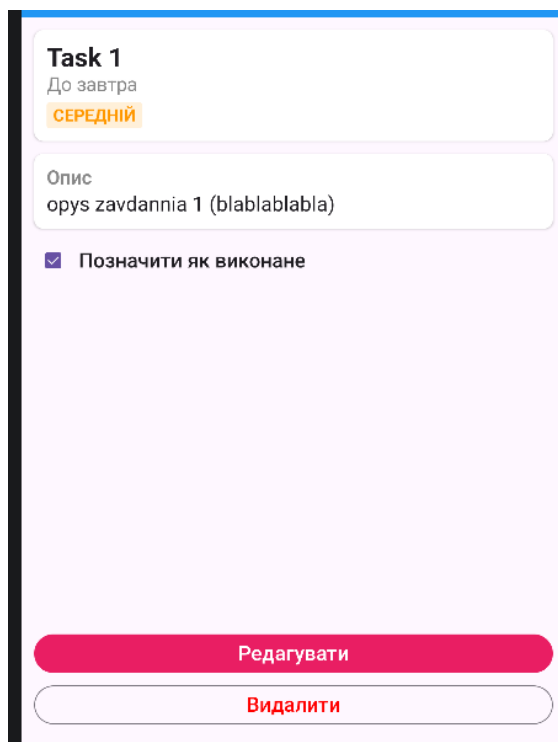


Рисунок 9, 10 – Видалення першого завдання зі списку завдань

ВИСНОВКИ

Усі вимоги лабораторної роботи виконано. Застосунок повністю переведено на багаторівневу архітектуру з розділенням відповідальності. Дані зберігаються у локальній SQLite-базі даних за допомогою ORM Room та не втрачаються після закриття програми.